

CONSULTING PERSONALITY PRACTICE

Personality Test 1 & 2

Requirements Specification & High-Level Design

Version 2.1 · 11 June 2026

Standalone React + Vite, YAML-driven (Test 1 & Test 2).

Covers: requirements, architecture, schema, and scoring algorithms (forced-choice + Likert + multi-select, sten report, self-vs-others consistency).

CCA Personality Prep — Requirements Specification

Version: 2.1 · **Date:** 2026-06-11 · **Status:** implemented (standalone React/Vite app)

v2 supersedes v1. The product was rebuilt as a standalone **React + Vite + Tailwind** single-page app that is **YAML-driven** (mirroring the existing `cca-numerical` quiz platform), replacing the earlier Next.js prototype. v2.1 adds the SHL-style **forced-choice** format and a **sten-based report**, and neutralises product branding.

1. Purpose & Scope

A web app for practising the two **personality sections** of an SHL-style consulting career assessment. Quizzes are authored as YAML files; the app renders them, scores answers into a **7-trait profile**, presents an SHL-style **sten report**, and analyses the **consistency between Test 1 (self-perception) and Test 2 (how others see you)** — the behaviour the assessment screens on.

In scope (v2.1):

- YAML-driven quizzes loaded via `?quiz-id=<file>.yaml` from `public/quizzes/`.
- Question types: `forced-choice` (SHL OPQ-style most/least), `likert` (1–5 agreement), `multi-select` (choose up to N), `single-select`. Types may be mixed in one quiz.
- In-app **trait scoring** (ipsative forced-choice; Likert with reverse-scoring; per-option weights).
- **SHL-style report:** sten scores (1–10) on a profile chart with the population-average band, percentile equivalents, and original per-trait narrative interpretation.
- Per-test profile **and** Test 1 ↔ Test 2 consistency analysis (sten gaps + extreme-answer flags).
- Local persistence (`localStorage`) so the cross-test comparison works without a backend.
- Optional timer (practice/test mode) when a quiz defines one; personality sections are untimed.

Out of scope (v2.1): user accounts / server database; the numerical question types (`fill-box` , `chart-adjust` , etc.) and correctness scoring used by the `cca-numerical` sets; payment; SHL's proprietary norm group (the sten norming is illustrative, not an official SHL norm comparison).

Branding & disclaimer: the product is named neutrally ("Consulting Personality Practice"); BCG/SHL/OPQ/CCA appear only descriptively. A prominent disclaimer states it is **not affiliated with or endorsed by BCG or SHL**, that those names are trademarks of their owners, and that all items/interpretations are original practice material.

2. Actors

Actor	Description
Candidate	Takes a quiz, sees their trait profile + consistency analysis.
Content author (Eric)	Drops a YAML file into <code>public/quizzes/</code> and redeploys; owns all question content.
Operator	Builds and hosts the static site (e.g. Vercel).

3. Domain Glossary

- **Trait** — one of 7 measured dimensions: `leadership`, `communication`, `adaptability`, `empathy`, `teamwork`, `time_management`, `cognitive`.
- **Quiz** — a YAML file describing one test (metadata + questions).
- **Question** — `likert`, `multi-select`, or `single-select`; each carries trait tag(s).
- **Trait Profile** — a vector of 7 scores in `[0,100]`.
- **Consistency Score** — `[0,100]` alignment between a Test 1 and Test 2 profile.

4. Functional Requirements

4.1 Quiz loading

- **FR-L1** The app reads `quiz-id` from the URL query and fetches `/quizzes/<quiz-id>`; default `personality-test-1.yaml`.
- **FR-L2** The YAML is parsed and **validated**; invalid files show a clear, specific error (which question, what's wrong) rather than failing silently.
- **FR-L3** Question content is authored as raw HTML in the `text` field and rendered as such (author-controlled, not user input).

4.2 Question types

- **FR-Q0** `forced-choice` — a block of ≥ 3 statements, each a *different* trait; the candidate marks the one **most** and the one **least** like them (a statement cannot be both). The SHL OPQ-style ipsative format.
- **FR-Q1** `likert` — one statement, answered on a 1–5 agreement scale (single-select); maps to one `trait`, optional `reverse`.
- **FR-Q2** `multi-select` — a prompt with options; the candidate selects up to `maxSelections`; each option maps to a `trait` (+ optional `weight`).
- **FR-Q3** `single-select` — a prompt with options; exactly one chosen; each option maps to a `trait`.

4.3 Taking a quiz

- **FR-T1** Questions are presented one per screen with a progress bar and position indicator.
- **FR-T2** Navigation: Back/Next; Next is disabled until the current question is answered; the last screen offers Finish, enabled only when all questions are answered.

- **FR-T3** Multi-select enforces `maxSelections / minSelections` ; forced-choice requires both a *most* and a *least* (and they must differ).
- **FR-T4** A start screen shows title/section, a prominent disclaimer, and (if timers are defined) a Practice vs. Test choice; personality quizzes (timer 0) start untimed.
- **FR-T5** If a non-zero timer is active and expires, the quiz auto-submits.
- **FR-T6** Keyboard support: Likert items accept keys 1–5; Enter/→ advances, ← goes back.

4.4 Scoring & results

- **FR-S1** On finish, the app computes the trait profile in-app (algorithm fixed; see DESIGN §4).
- **FR-S2** Results present an **SHL-style report**: each trait as a **sten score (1–10)** on a profile chart with the population-average band shaded, plus percentile and band, and an original narrative interpretation per trait.
- **FR-S3** If a profile for the *other* test exists (this session or a prior one), results show the **consistency analysis**: overall score, trait-by-trait sten-gap table, and extreme-answer flags, with a plain-language summary.
- **FR-S4** With only one test taken, results prompt the user to take the other to unlock the comparison, linking to it.
- **FR-S5** Each test's latest profile persists in `localStorage` , keyed by test number.
- **FR-S6** The report states that sten norming is illustrative, not SHL's proprietary norm group.

4.5 Content authoring

- **FR-C1** Adding/editing a quiz requires only a YAML file in `public/quizzes/` ; no code changes.
- **FR-C2** The schema is backward-compatible with the `cca-numerical` metadata keys (`title` , `subtitle` , `section` , `quizNumber` , `practiceMode` , `testMode`).

5. Non-Functional Requirements

ID	Requirement
NFR-1 (No backend)	Pure static SPA; runs from any static host. State lives in the browser.
NFR-2 (Determinism)	Scoring is a pure function of answers + quiz; identical inputs → identical profile.
NFR-3 (Validation)	Malformed YAML never crashes silently; the user sees what to fix.
NFR-4 (Performance)	Single small bundle (~65 kB gzipped); instant scoring (O(#answers)).
NFR-5 (Responsive)	Usable on mobile and desktop widths.
NFR-6 (Maintainability)	Scoring isolated in <code>src/lib/scoring.ts</code> , framework-free and unit-testable.
NFR-7 (Authoring fidelity)	Same YAML conventions as <code>cca-numerical</code> so content tooling/workflow is shared.
NFR-8 (Privacy)	No accounts, no server, no third-party analytics; only local trait scores stored.

6. Constraints & Assumptions

- Real CCA item content is proprietary; the app models the **format**, not the items.
 - Cross-test consistency relies on `localStorage`, so it is per-browser (clearing storage resets it). Acceptable given the no-backend constraint.
 - Node 18+ / modern evergreen browsers.
-

7. Acceptance Criteria (v2) — passing

- Loads & validates `personality-test-1.yaml` / `-2.yaml`; bad YAML shows a specific error. 
 - `likert` (with reverse) and `multi-select` (with `maxSelections`) render and are answerable. 
 - Finishing produces a 7-trait profile (all values 0–100); reverse + multi-select scoring verified numerically. 
 - Taking both tests shows the consistency score, gap table, and extreme flags. 
 - Production build compiles and type-checks clean. 
-

8. Roadmap (not in v2)

Item	Summary
Full content	Expand to the real ~76 (Test 1) + ~15 (Test 2) items (author-supplied).
Pre-test coaching	Briefing + per-trait "what good looks like" guidance.
Progress trend	Track consistency across attempts over time.
Answer review	Let candidates revisit/change answers before submitting.

CCA Personality Prep — High-Level Design

Version: 2.1 · **Date:** 2026-06-11 · Companion to [REQUIREMENTS.md](#)

v2 = standalone React + Vite + Tailwind SPA, YAML-driven, no backend. v2.1 adds the forced-choice algorithm and the SHL-style sten report.

1. Architecture Overview

A static single-page app. There is no server: the browser fetches a YAML quiz, renders it, scores answers locally, and persists results in `localStorage`.

Browser (SPA)

```
App.tsx (phase machine: loading→running→results)
├── lib/loadQuiz.ts —fetch→ /quizzes/<id>.yaml
│   (js-yaml parse + validate)
├── components/Quiz.tsx
│   Likert / MultiSelect / SingleSelect inputs
│   Timer (only if quiz defines one)
├── lib/scoring.ts (pure: scoreQuiz, buildFeedback)
├── components/Results.tsx + TraitBar.tsx
└── lib/storage.ts → localStorage (per-test profile)
```

served as static files (Vite build → dist/)

1.1 Module responsibilities

Module	Responsibility
src/App.tsx	Top-level state machine: load → run → results; reads <code>quiz-id</code> .
src/lib/loadQuiz.ts	Fetch + <code>js-yaml</code> parse + schema validation with actionable errors.
src/lib/traits.ts	The 7 traits, lookup maps, <code>isTraitId</code> guard.
src/types.ts	Quiz / Question / Answer types mirroring the YAML schema.
src/lib/scoring.ts	All algorithms — <code>scoreQuiz</code> , <code>buildFeedback</code> . Pure, deterministic.
src/lib/storage.ts	Persist/read the latest profile per test (<code>localStorage</code>).
src/components/Quiz.tsx	Start screen, per-question rendering, navigation, finish.
src/components/Results.tsx	Trait profile + consistency analysis + extreme flags.
src/components/TraitBar.tsx	Labelled bars per trait (self vs. others colouring).
src/components/Timer.tsx	Countdown, used only for timed quizzes.
src/components/Html.tsx	Renders author-controlled HTML from YAML <code>text</code> .

1.2 Key design choices

- **YAML-driven, content-out-of-code** — authors add quizzes by dropping files in `public/quizzes/` , matching the `cca-numerical` workflow. The app ships the engine + UI only.
- **Algorithm in app, content in YAML** — the YAML tags each item with its trait(s); the scoring math lives in `scoring.ts` (a single source of truth).
- **Pure scoring functions** — no I/O, fully deterministic, unit-testable.
- **No backend** — cross-test consistency uses `localStorage` ; simplest thing that satisfies the requirement without servers or accounts.

2. Data / Schema Model

YAML quiz (personality), backward-compatible with the numerical sets' metadata:

```
title, subtitle, section, quizNumber    # display metadata (shared convention)
test: 1 | 2                             # drives the consistency comparison
scoring: trait                           # vs. "correctness" (numerical sets)
scale: likert5                            # 1=Strongly disagree ... 5=Strongly agree
practiceMode, testMode                   # timer seconds; 0/absent = untimed
format: forced-choice | likert | multi-select # informational; types may be mixed
questions:
  - { id, type: forced-choice, text?, statements: [{text, trait}] } # ≥3, distinct traits
  - { id, type: likert, trait, reverse?, text }
  - { id, type: multi-select, maxSelections?, minSelections?, text, options: [{text, trait,
  - { id, type: single-select, text, options: [{text, trait, weight?}] }
```


Persisted state (localStorage): cca_profile_test1 / cca_profile_test2 → { profile: TraitProfile, at: epochMs }. Only the computed profile is stored, so the comparison survives separate sittings.

3. Runtime Flow

1. App reads quiz-id , calls loadQuiz → parse + validate.
 2. Quiz renders the start screen, then one question per screen; answers held in component state.
 3. On finish → scoreQuiz(quiz, answers) → save to localStorage → show Results .
 4. Results loads the other test's stored profile; if present → buildFeedback → consistency view.
-

4. Algorithms (src/lib/scoring.ts)

Notation: 7 traits indexed by `t` ; profiles map `trait → score ∈ [0,100]` . All answer types are projected onto a common **contribution scale** `[0,1]` , then a trait's score is the mean of its contributions $\times 100$. This unifies Likert and select-style items so they can coexist in one quiz.

4.1 Per-type contributions

```
forced-choice (block of statements; answer = {most, least}):  # ipsative
  for each statement s at index i:
    contribution = (i == most) ? 1 : (i == least) ? 0 : 0.5
    → added to s.trait
  # over many blocks: a trait often "most" trends high, often "least" low,
  # unpicked ~0.5 – the SHL relative (ipsative) profile.

likert (value v ∈ 1..5):
  effective = reverse ? (6 - v) : v          # reverse-score negatively-worded items
  contribution = clamp01( (effective - 1) / 4 )  # 1→0.0, 3→0.5, 5→1.0
  → added to the item's single `trait`

single-select (chosen option o):
  contribution = clamp01( o.weight ?? 1 )      # → o.trait

multi-select (each option o at index i):
  contribution = selected(i) ? clamp01(o.weight ?? 1) : 0  # → o.trait
  # every option is a binary item for its trait: chosen = endorse, unchosen = 0
```

4.2 Aggregation

```
for each trait t:
  xs = all contributions routed to t
  profile[t] = xs.length ? round( mean(xs) * 100 ) : 50  # 50 = neutral when unmeasured
```

- **Complexity:** $O(\text{\#questions} + \text{\#options})$.
- **Why mean (not sum):** keeps every trait on a comparable 0–100 scale regardless of how many items measure it — authors don't have to balance item counts exactly.

- **Worked example.** `time_management` measured by: a normal Likert at 4 ($\rightarrow 0.75$), a reverse Likert at 4 (effective 2 $\rightarrow 0.25$), and an unselected multi-select option ($\rightarrow 0$). Mean = $(0.75+0.25+0)/3 = 0.33 \rightarrow 33$. (Verified against the live engine.)

4.3 Consistency analysis (`buildFeedback(p1, p2)`)

```
for each trait t: gap[t] = |p1[t] - p2[t]|
gaps_sorted    = traits by gap, descending
avgGap         = mean(gap)
consistency    = max(0, round(100 - avgGap))    # 0..100, higher = more aligned
inconsistencies = { t : gap[t] > 25 }           # GAP_FLAG = 25
extremes       = { (t,test) : score ≥ 88 (high) or ≤ 12 (low) } # on BOTH profiles
```

Summary is rule-based: ≥ 80 and no inconsistencies $\rightarrow$ "strong, coherent"; any inconsistencies $\rightarrow$ name the top 1–2 gap traits; else $\rightarrow$ "tighten the largest gaps"; append an "avoid extremes" note if any extremes exist.

Thresholds (centralised in `scoring.ts`): `100 - avgGap` is intuitive and bounded; `GAP_FLAG = 25` (~quarter scale) marks genuine self-vs-others disagreement; `HIGH/LOW = 88/12` flag the top/bottom ~12% spikes BCG penalises. **Complexity:** $O(7)$.

4.4 SHL-style report — sten standardisation (`src/lib/sten.ts`)

The report converts each 0–100 trait score into a **sten** (Standard Ten: mean 5.5, sd 2, range 1–10) — the SHL reporting metric — plus a percentile.

```
z          = (score - POP_MEAN) / POP_SD          # POP_MEAN = 50, POP_SD = 20 (illustrative)
sten       = clamp(round(5.5 + 2·z), 1, 10)
percentile = clamp(round( $\Phi(z)$  · 100), 1, 99)      #  $\Phi$  = standard normal CDF (A&S approx)
band       = Low | Below average | Average | Above average | High # by sten range
```

The profile chart shades stens **4–7** as the population-average band and marks each trait's sten. Per-trait narrative text (`src/lib/narratives.ts`) is selected by band ($\text{low} \leq 4$ / avg / $\text{high} \geq 8$) and is original wording.

Norming caveat: `POP_MEAN/POP_SD` define an *illustrative* population, not SHL's proprietary norm group (which is not public). The sten metric, chart, percentile, and banded interpretation mirror SHL's report **format**; the numbers indicate relative standing, not an official SHL norm comparison. This is stated in the UI.

5. Validation & Error Handling

`loadQuiz` rejects with a specific message when: the file is missing/empty, has no `questions`, a likert item lacks a valid `trait`, or a select item has < 2 options or an option with an invalid `trait`. Errors render in a visible panel naming the offending question — authoring feedback, not a blank screen.

6. Build & Deployment

- **Dev:** `npm run dev` → `http://localhost:5173/?quiz-id=personality-test-1.yaml` .
 - **Build:** `npm run build ( tsc -b && vite build )` → static `dist/` .
 - **Host:** any static host (e.g. Vercel), same as `cca-numerical` .
 - **Add content:** drop a YAML in `public/quizzes/` , redeploy.
-

7. Testing Strategy

Layer	Approach
Scoring	Pure-function checks: known answers → expected profile; reverse + multi-select + empty-trait (→50) edge cases. (Verified on the real YAML.)
Loader	Valid file parses; malformed files produce the expected validation error.
UI	Manual: render likert/multi, enforce <code>maxSelections</code> , gating of Next/Finish, results + consistency render.
E2E (planned)	Headless run: load both YAMLs, answer, assert consistency view.

